

App.py

Gestura is a real-time sign language interpretation system that captures hand gestures through a webcam, extracts hand landmarks using MediaPipe, classifies gestures using a trained Random Forest model, displays interpreted text, and converts it into speech using text-to-speech.

This file is responsible for:

- GUI (Tkinter)
- Live camera feed
- Hand landmark detection
- Real-time prediction
- Text formation
- Speech output

◆ Libraries Used & Why

`import tkinter as tk`

`from tkinter import ttk, messagebox`

→ GUI framework for desktop application

`import cv2`

`from PIL import Image, ImageTk`

→ Webcam handling + displaying frames in Tkinter

`import pickle`

`import numpy as np`

→ Load trained ML model + numerical processing

`import mediapipe as mp`

→ Real-time hand landmark detection

`import threading`

`import time`

→ Prevent GUI freezing, control detection speed

`import pyttsx3`

→ Offline text-to-speech conversion

◆ MediaPipe Setup

```
mp_hands = mp.solutions.hands
```

```
mp_drawing = mp.solutions.drawing_utils
```

```
mp_drawing_styles = mp.solutions.drawing_styles
```

Used for:

- Hand detection
- Drawing landmarks
- Visual feedback on camera feed

✖ CLASS 1: SplashScreen

Purpose:

Displays a **professional intro screen** before main UI loads.

Key Features:

- Custom logo drawing using Canvas
- Animated “Loading...” text
- Developer names
- Auto-close after 5 seconds

```
self.splash.overrideRedirect(True)
```

→ Removes title bar → modern look

```
self.splash.after(duration, self.close_splash)
```

→ Automatically launches main app

📌 Viva Point:

Splash screen improves user experience and hides initial loading delay.

✖ CLASS 2: AboutWindow

Purpose:

Displays project and team details.

Includes:

- Student names & PRNs

- College name
- Guide name

`tk.Toplevel(parent)`

→ Creates a child window without blocking main app

✦ **Viva Point:**

This window is non-modal and improves project presentation.

✦ **CLASS 3: GesturaApp (MAIN APPLICATION)**

This is the **heart of the system** ❤️

♦ **__init__() – Application Initialization**

`self.root.withdraw()`

→ Hides main window until splash ends

Application State Variables

`self.camera_running`

`self.model`

`self.classes`

`self.detection_speed`

`self.confidence_threshold`

`self.detected_text`

Used to:

- Control camera
 - Control detection frequency
 - Filter weak predictions
 - Build sentence output
-

♦ **Text-to-Speech Setup**

`self.tts_engine = pyttsx3.init()`

✓ Offline

✓ No internet

✓ Platform-independent

◆ **load_model()**

with open('./models/sign_language_model.pkl', 'rb') as f:

Loads:

- Trained Random Forest model
- Class labels

📌 **Why pickle?**

Preserves trained parameters without retraining.

◆ **create_ui()**

Builds the Entire Interface

Left Panel:

- Live camera feed
- Start / Stop buttons
- About Us

Right Panel:

- Interpreted text
- Speak / Backspace / Clear buttons
- Speed & confidence sliders
- Current prediction display

📌 **Viva Line:**

UI is divided for clarity: visual input on the left, interpreted output on the right.

◆ **Speed & Confidence Sliders**

self.detection_speed

self.confidence_threshold

Slider	Purpose
--------	---------

Speed	Controls prediction frequency
-------	-------------------------------

Confidence	Filters weak predictions
------------	--------------------------

📌 **Prevents:**

- Duplicate letters
- Noisy predictions

♦ **start_camera()**

```
self.cap = cv2.VideoCapture(0)
```

- ✓ Opens webcam
- ✓ Starts detection thread
- ✓ Prevents UI freezing

```
threading.Thread(target=self.detect_signs, daemon=True)
```

🔴 **Viva Point:**

Threading allows real-time processing without blocking the GUI.

♦ **stop_camera()**

- ✓ Stops webcam
- ✓ Resets UI
- ✓ Releases hardware safely

♦ **detect_signs() (REAL-TIME LOOP)**

MOST IMPORTANT METHOD 🔥

```
with mp_hands.Hands(static_image_mode=False):
```

Parameter	Reason
static_image_mode=False	Video tracking
min_tracking_confidence	Stable landmark tracking

Steps:

1. Capture frame
 2. Flip for mirror view
 3. Detect hand landmarks
 4. Draw landmarks
 5. Predict sign at fixed interval
 6. Update UI safely
-

♦ **predict_sign()**

`self.model.predict()`

`self.model.predict_proba()`

Extracts:

- Predicted class
- Confidence score

if confidence >= self.confidence_threshold:

→ Only meaningful predictions are accepted

♦ **add_to_text()**

- ✓ Prevents repeated letters
 - ✓ Builds sentence word-by-word
 - ✓ Stores history for backspace
-

♦ **Text Controls**

Method	Function
<code>speak_text()</code>	Converts text → speech
<code>backspace_text()</code>	Removes last sign
<code>clear_text()</code>	Clears sentence

♦ **on_closing()**

Ensures:

- Camera released
- Threads stopped
- App exits cleanly

✦ **Very important for viva**

♦ **main() Function**

`root = tk.Tk()`

`app = GesturaApp(root)`

root.mainloop()

Entry point of application.

VIVA QUESTIONS & ANSWERS (EXTREMELY IMPORTANT)

Q1. What is Gestura?

Answer:

Gestura is a real-time sign language interpretation system that converts hand gestures into text and speech using computer vision and machine learning.

Q2. Why MediaPipe instead of image processing?

Answer:

MediaPipe provides accurate hand landmarks, reducing input complexity and improving real-time performance.

Q3. Why use threading?

Answer:

To prevent GUI freezing while performing continuous camera capture and prediction.

Q4. Why Random Forest for prediction?

Answer:

Because the model works efficiently on structured landmark data and gives fast predictions suitable for real-time use.

Q5. What is confidence threshold?

Answer:

It filters out low-confidence predictions to avoid incorrect gesture recognition.

Q6. Why convert OpenCV frames to PIL?

Answer:

Tkinter cannot directly display OpenCV images; PIL acts as a bridge.

Q7. Why flip the camera image?

Answer:

To provide a natural mirror-like user experience.

Q8. Can this system work offline?**Answer:**

Yes. Both MediaPipe and pytsx3 work offline.

Q9. What are limitations?**Answer:**

- Single hand only
 - Static gestures only
 - Sensitive to lighting
-

Q10. Future Improvements?**Answer:**

- Sentence-level gestures
 - LSTM for dynamic signs
 - Multi-hand recognition
 - Mobile app deployment
-

** FINAL EXAMINER-READY SUMMARY (MEMORIZE THIS)**

“Gestura is a real-time sign language interpreter that uses MediaPipe for hand landmark extraction, a Random Forest classifier for gesture recognition, and Tkinter for GUI interaction. The system converts gestures into readable text and audible speech, enabling effective communication for hearing-impaired users.”